

Figure 4: DNN block diagram

Defining DNN

In Figure 2, X_i represents the input neuron while $f(y)$ represents the output neuron, while layers are called the input and output layer, respectively.

In DNN, there are multiple hidden layers of units between the input and output layers, as shown in Figure 4.

Let us see how the theory discussed so far is used practically, in the following text.

Training a network

There are two stages involved in a DNN:

1. Training the network
2. Testing the network for how well it has been trained

Before we move on, let's get familiar with the mathematical terms and definitions that we will use, going forward.

To begin with, you can refer to Box 1 for all the relevant formulae.

Next, the gradient descent is used to find the local minima of a function (Figure 5).

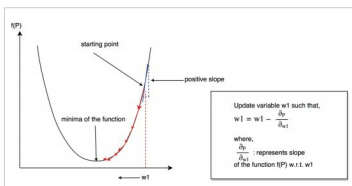


Figure 5: Gradient descent

The gradient descent works as follows:

- The given performance function $f(p)$ is dependent on w_1 .
- We have to calculate the minima of the function (i.e., the value for which the function is minimum).
- We randomly initialise the value of w_1 to get the starting point.
- Then we calculate the slope of the function with respect to w_1 and update the weight as follows:

$$w_1 = w_1 - \frac{\partial p}{\partial w_1}$$

- As per Figure 5, the slope is positive, w_1 is decreasing, and we are going in the correct (i.e., downward) direction. Back propagating errors occur from the output to the input layer.

Now, let's look at how to implement a three-layer DNN.

Understanding problem statements

In Figure 6, we have four images in binary format given as inputs to the network. Each image is 3 bits long. With respect to each binary input, we have the input neuron defined. So there are three input neurons in the input layer. There are four neurons in the hidden layer. And one output neuron.

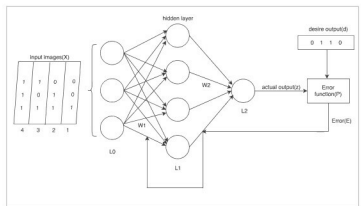


Figure 6: Three-layer DNN

With respect to each image, there is a predefined desired output (d).

Assume that Images 1 and 4 are of digit 0 with the desired output-binary 0; and Images 2 and 3 are of digit 1 with the desired output-binary 1.

There are 12 weights in Layer L1 and four in Layer L2 for connection between the two layers.

As our input and output is fixed, we need to find weights in Layer L1- w_1 and weights in Layer L2- w_2 such that our actual output (z) is equal to the desired output (d).

Figure 6 can be drawn as a block diagram as shown in Figure 7 to derive the required equations.

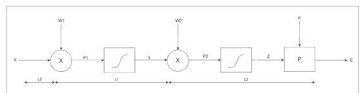


Figure 7: Three-layer DNN block diagram

Box 2 gives different equations for DNNs.

Steps for implementing a three-layer DNN

1. Initialise the weights w_1 randomly. Dot product of input- x and weights w_1 produces P_1 , which acts as input to the